



Procesador RISC-V con Arquitectura Pipelined (Segmentada)

Informe Técnico de Diseño e Implementación

Curso: Arquitectura de Computadoras 2026-1
Sección: 4
Universidad: Universidad de Ingeniería y Tecnología (UTEC)
Repositorio: github.com/Auky216/Microarchitecture-Pipelined

Integrantes:

Isaac Emanuel Javier Simeón Sarmiento

isaac.simeon@utec.edu.pe

Alexandro Martin Chamocho Gumbi Gutierrez

alexandro.chamocho@utec.edu.pe

Adrian Antonio Auqui Perez

adrian.auqui@utec.edu.pe

18 de junio de 2026

Índice

1. Teoría del Procesador Pipeline: Datapath y Control (0.5 pts)	3
1.1. ¿Qué es un Procesador Pipelined?	3
1.2. Las 5 Etapas del Pipeline	3
1.3. Diagrama Temporal del Pipeline	3
1.4. ISA Implementada: RISC-V RV32I	4
2. Implementación de Instrucciones (Cuadro 1) (1.0 pts)	4
2.1. Descripción de Cambios en el Código	4
2.2. Diagrama de Datapath Final (Básico)	5
2.3. Los Registros Pipeline	6
2.3.1. ¿Qué son y por qué son necesarios?	6
2.3.2. Tres Tipos de Flip-Flops	7
2.3.3. ¿Por qué IF/ID usa flopenrc y MEM/WB usa flopr?	7
2.4. Módulo Top-Level: riscv_pipeline	7
2.5. Etapa 1: Fetch (IF)	8
2.5.1. Componentes Activos	8
2.5.2. Pseudocódigo	8
2.6. Etapa 2: Decode (ID)	9
2.7. Componentes Activos	9
2.7.1. Unidad de Control	9
2.7.2. Pseudocódigo de Decode	10
3. Etapa 3: Execute (EX)	10
3.1. Componentes Activos	11
3.2. Pseudocódigo Completo del Execute	11
4. Etapas Memory (MEM) y Writeback (WB)	12
4.1. Componentes Activos	13
4.2. Pseudocódigo de ambas etapas	13
5. Programa ISA sin Dependencias (1.0 pts)	14
5.1. Código Ensamblador	14
5.2. Waveform del Programa 1	14
5.3. Mostrar con Resultados Intermedios que el Programa Funciona Correctamente	15
6. Teoría de Hazard Unit y Evidencia de Fallos (0.5 pts)	15
6.1. Teoría de Control de Riesgos en Procesadores Segmentados	15
6.2. Evidencia: Sin Hazard Unit los Programas Fallan	15
6.2.1. Programa 2 SIN Hazard Unit (Forwarding)	16
6.2.2. Programa 3 SIN Hazard Unit (Stalling)	16
6.2.3. Programa 4 SIN Hazard Unit (Flushing)	16
7. Implementación del Hazard Unit (1.0 pts)	17
7.1. Descripción de Cambios en el Código	17
7.2. Mecanismo 1: Forwarding (Reenvío de Datos)	17

7.3. Mecanismo 2: Stall (Load-Use Hazard)	18
7.4. Mecanismo 3: Flush (Control Hazard por Saltos)	18
7.5. Especificación de Señales de Control de Riesgos	18
7.6. Diagrama de Datapath Final (con Hazard Unit)	19
8. Programas de Prueba (CON Hazard Unit) (1.0 pts)	20
8.1. Programa 2: Forwarding	20
8.2. Programa 3: Stalling	20
8.3. Programa 4: Flushing	21
9. Jerarquía de Archivos	21
10. Conclusiones	22

1 Teoría del Procesador Pipeline: Datapath y Control (0.5 pts)

1.1 ¿Qué es un Procesador Pipelined?

Un procesador **pipelined** (segmentado) divide la ejecución de cada instrucción en varias etapas independientes que trabajan en paralelo. Mientras una instrucción está siendo decodificada, la siguiente ya está siendo leída de la memoria, y la anterior ya está realizando su operación aritmética.

Teoría del Paralelismo a Nivel de Instrucción (ILP): El pipelining es la forma más pura de ILP. En un procesador **de ciclo único**, cada instrucción tarda un ciclo completo (cuyo reloj se calibra según la instrucción más lenta, como `lw`). La latencia total y el periodo del reloj limitan drásticamente la frecuencia (MHz/GHz). En contraste, el pipeline divide ese largo camino combinacional en **5 etapas** más cortas separadas por registros.

Esto reduce drásticamente el periodo del reloj y aumenta la frecuencia. Idealmente, el **throughput** (rendimiento) se incrementa por un factor igual al número de etapas ($k = 5$). Aunque la *latencia* de una instrucción individual no mejora (incluso empeora ligeramente por el retardo de los registros), el hecho de que termine **una instrucción por cada ciclo de reloj** hace que el procesador sea inmensamente más rápido a gran escala.

1.2 Las 5 Etapas del Pipeline

1. **Fetch (IF):** Lectura de la instrucción desde la memoria de instrucciones.
2. **Decode (ID):** Decodificación de la instrucción, lectura de registros y generación de señales de control.
3. **Execute (EX):** Ejecución de la operación aritmético-lógica en la ALU.
4. **Memory (MEM):** Acceso a la memoria de datos.
5. **Writeback (WB):** Escritura del resultado final en el banco de registros.

1.3 Diagrama Temporal del Pipeline

El siguiente diagrama muestra cómo 5 instrucciones se solapan en el pipeline:

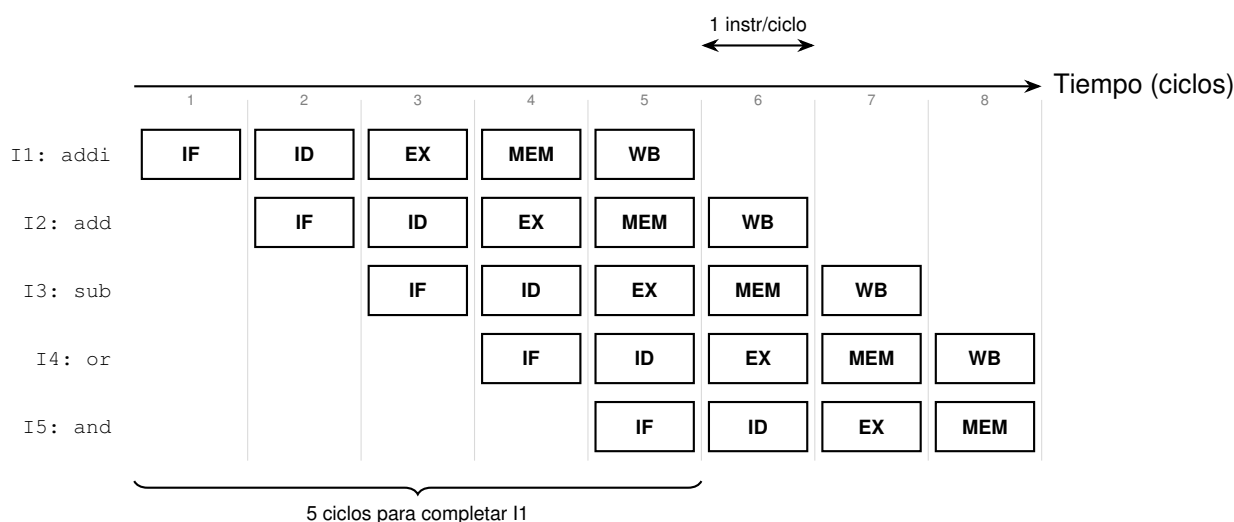


Figura 1: Diagrama temporal: cada instrucción tarda 5 ciclos, pero se completa 1 instrucción por ciclo.

1.4 ISA Implementada: RISC-V RV32I

Tipo	Instrucciones	Opcode	Ejemplo
R-type	add, sub, and, or, sll, slt	0110011	add x4, x2, x3
I-type	addi, lw, ori, slti	0010011 / 0000011	addi x2, x0, 5
S-type	sw	0100011	sw x7, 100(x3)
B-type	beq	1100011	beq x4, x0, L
J-type	jal	1101111	jal x3, func

Tabla 1: Instrucciones soportadas por el procesador.

2 Implementación de Instrucciones (Cuadro 1) (1.0 pts)

2.1 Descripción de Cambios en el Código

Para transformar la arquitectura de ciclo único a una segmentada (pipelined) que soporte el conjunto de instrucciones del Cuadro 1 sin degradar la correctitud, se realizaron los siguientes cambios estructurales en el código Verilog:

1. **Segmentación de Módulos en el Datapath:** La lógica combinacional monolítica se dividió en 5 etapas independientes ubicadas en la carpeta `Datapath/`:
 - `fetch.v`: Controla el Program Counter (PC) y accede a la memoria de instrucciones `imem.v`.
 - `decode.v`: Decodifica operandos, lee del banco de registros (`regfile.v`) y extiende inmediatos.

- `execute.v`: Procesa operaciones mediante la ALU y calcula la condición y dirección del branch.
 - `memory.v`: Maneja la lectura asíncrona y escritura síncrona en la memoria de datos `dmem.v`.
 - `writeback.v`: Selecciona el dato final a guardar en el Register File.
2. **Control Pipelined (Control Pipeline)**: Las señales generadas por la unidad de control en Decode (`controller.v`) se segmentaron agregando registros pipeline. Las señales viajan junto con la instrucción por el datapath (`RegWriteD` → `RegWriteE` → `RegWriteM` → `RegWriteW`).
 3. **Propagación de Señales de Instrucción (Debug)**: Se implementó el tránsito de las instrucciones completas de 32 bits por cada etapa (`InstrD`, `InstrE`, `InstrM`, `InstrW`). Esto permite visualizar en el testbench y en GTKWave exactamente qué instrucción se encuentra en ejecución o acceso a memoria en cada ciclo.
 4. **Dimensionamiento del Registro del Pipeline**: Al inyectar la señal de instrucción de 32 bits en las etapas, se reajustaron los anchos de los flip-flops del pipeline (`floprr/flopenrc`):
 - **Registro ID/EX**: De 186 bits a 218 bits (agregando `InstrD`).
 - **Registro EX/MEM**: De 105 bits a 137 bits (agregando `InstrE`).
 - **Registro MEM/WB**: De 104 bits a 136 bits (agregando `InstrM`).
 5. **Control del ALU a 4 bits**: En la unidad de control (`controller.v`), se configuró un bus de 4 bits para la ALU. Esto permite discriminar correctamente la instrucción `add` de `sub` (mediante el bit de `funct7`) y asegurar que el sumador/restador interno funcione sin conflictos en saltos y cargas.

2.2 Diagrama de Datapath Final (Básico)

A continuación, se presenta el esquema del datapath segmentado básico para la ejecución de instrucciones secuenciales (sin incluir la Hazard Unit):

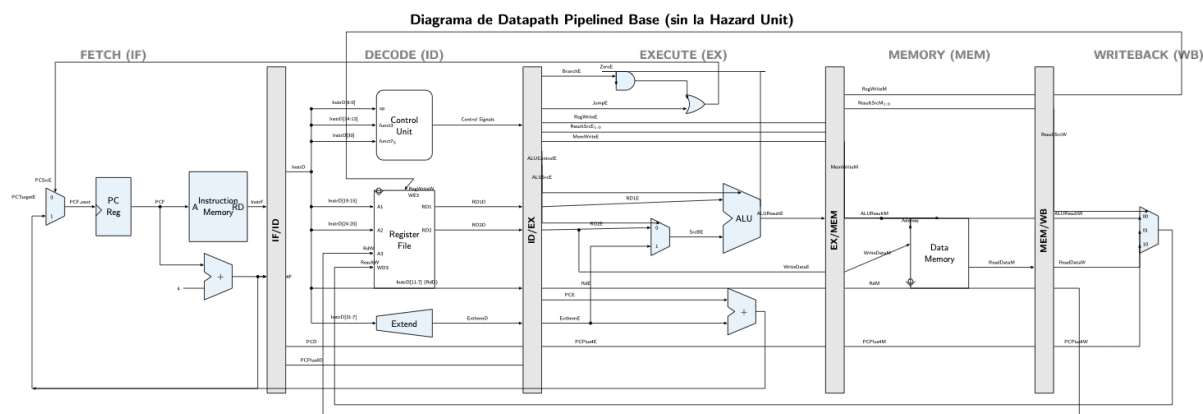


Figura 2: Diagrama del Datapath Pipelined básico del procesador (sin la Hazard Unit).

El datapath básico funciona dividiendo el camino crítico combinacional de la arquitectura de ciclo único en cinco etapas más cortas y balanceadas de forma paralela. En este diseño sin la Hazard Unit, el procesador opera correctamente bajo la suposición de que no existen dependencias de datos ni de control entre instrucciones consecutivas (o que estas se resuelven agregando instrucciones `nop` manuales en el ensamblador). Cada una de las etapas (IF, ID, EX, MEM, WB) ejecuta de forma concurrente una instrucción gracias a los registros de acoplamiento (IF/ID, ID/EX, EX/MEM, MEM/WB), los cuales retienen los operandos y las señales de control correspondientes. Esto permite completar la ejecución de una instrucción en cada ciclo de reloj (IPC ideal = 1) una vez que se ha llenado el pipeline.

2.3 Los Registros Pipeline

2.3.1 ¿Qué son y por qué son necesarios?

Los registros pipeline **separan físicamente** una etapa de la siguiente. Sin ellos, las señales se mezclarían. Se ubican entre cada par de etapas.

2.3.2 Tres Tipos de Flip-Flops

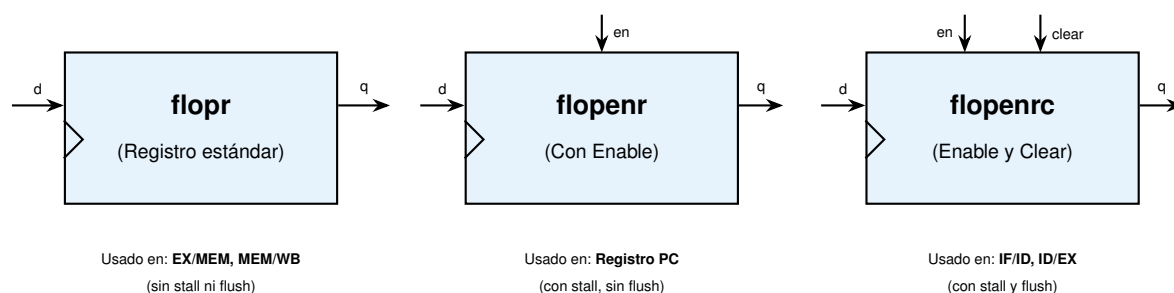


Figura 3: Los tres tipos de flip-flops usados. Observa el triángulo símbolo de la señal de reloj sincronizada por flanco.

2.3.3 ¿Por qué IF/ID usa flopenrc y MEM/WB usa flopr?

Las etapas tempranas (Fetch, Decode, Execute) son **especulativas**: las instrucciones pueden ser descartadas (flush) si un salto cambia el flujo, o detenidas (stall) si hay una dependencia de datos sin resolver.

Las etapas tardías (Memory, Writeback) ejecutan instrucciones que ya **pasaron todos los filtros**: ya se resolvieron los saltos y los riesgos. Nunca hay razón para detenerlas ni vaciarlas, de ahí que solo requieran un flopr básico.

```

1 // flopenrc: Tiene clk, reset, enable y clear
2 always @(posedge clk):
3     if (reset)      q = 0;
4     else if (clear) q = 0; // FLUSH: inserta burbuja NOP
5     else if (en)    q = d; // Avanza normalmente
6     // else if (!en): q no cambia (CONGELADO - STALL)

```

Listing 1: Comportamiento de los registros pipeline

2.4 Módulo Top-Level: riscv_pipeline

El módulo riscv_pipeline es el **ensamblaje principal** que conecta las 5 etapas con la Hazard Unit. Es puramente estructural, cableando todos los submódulos.

```

1 module riscv_pipeline(clk, reset):
2     // 1. Instanciar Fetch
3     fetchStage(clk, reset, StallF, PCSrcE, PCTargetE ...);
4
5     // 2. Instanciar Decode
6     decodeStage(clk, reset, StallD, FlushD, ...);
7
8     // 3. Instanciar Execute
9     executeStage(clk, reset, FlushE, ForwardAE, ForwardBE, ...);
10
11    // 4. Instanciar Memory
12    memStage(clk, reset, execute_signals ...);
13
14    // 5. Instanciar Writeback

```

```

15  wbStage(clk, reset, memory_signals ...);
16
17  // 6. Instanciar Hazard Unit
18  hazardUnit(Rs1D, Rs2D, Rs1E, Rs2E, RdE, RdM, RdW, ...);

```

Listing 2: Pseudocódigo del módulo top-level

2.5 Etapa 1: Fetch (IF)

2.5.1 Componentes Activos

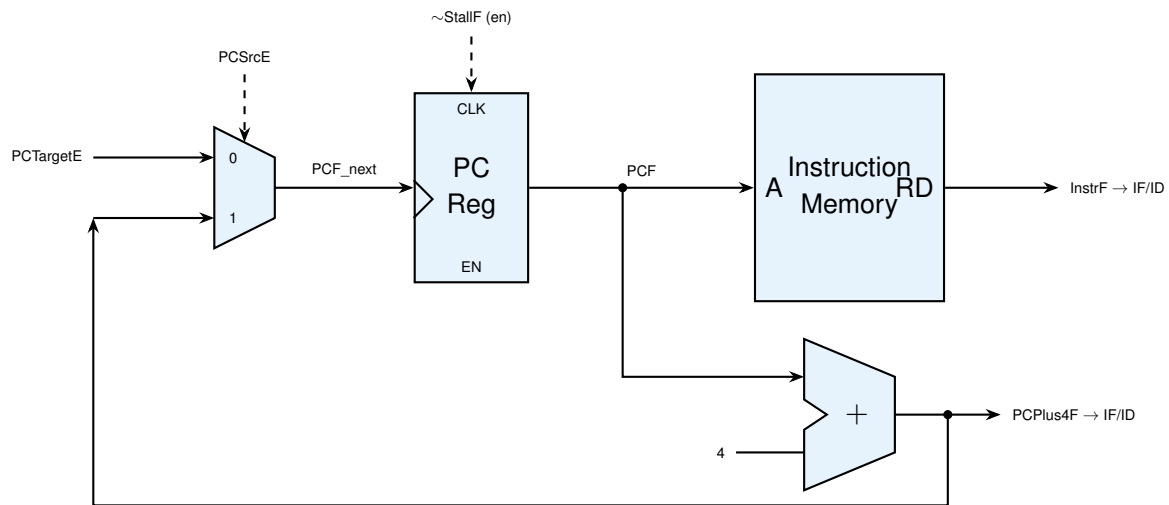


Figura 4: Etapa Fetch aislada con sus componentes operativos.

2.5.2 Pseudocódigo

```

1  FETCH(clk, reset, StallF, PCSrcE, PCTargetE):
2    // 1. Seleccionar el proximo PC
3    if (PCSrcE == 1):
4      PCF_next = PCTargetE;      // Salto tomado
5    else:
6      PCF_next = PCPlus4F;      // Secuencial (PC + 4)
7
8    // 2. Actualizar el PC (solo si no hay stall)
9    always @(posedge clk):
10     if (reset)      PCF = 0;
11     else if (!StallF) PCF = PCF_next;
12
13    // 3. Leer la instruccion de la memoria
14    InstrF = instruction_memory[PCF / 4];
15
16    // 4. Calcular PC + 4
17    PCPlus4F = PCF + 4;

```

Listing 3: Pseudocódigo de la etapa Fetch

2.6 Etapa 2: Decode (ID)

2.7 Componentes Activos

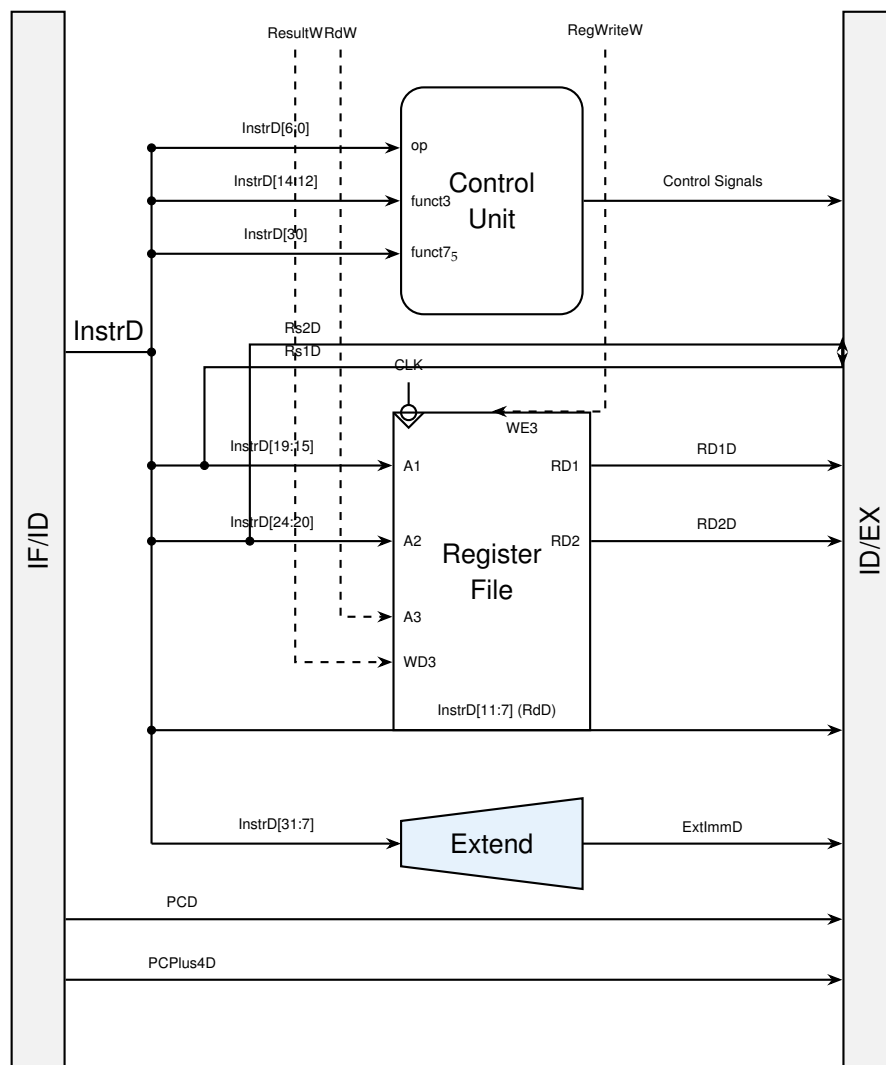


Figura 5: Etapa Decode aislada. Incluye el banco de registros y unidad de control.

2.7.1 Unidad de Control

La Unidad de Control recibe el opcode, `funct3` y `funct7` de la instrucción y genera todas las señales de control.

Tipo	Opcode	RegW	ImmSrc	ALUSrc	MemW	ResSrc	Branch	ALUOp
lw	0000011	1	00	1	0	01	0	00
sw	0100011	0	01	1	1	–	0	00
R-type	0110011	1	xx	0	0	00	0	10
I-type	0010011	1	00	1	0	00	0	10
beq	1100011	0	10	0	0	–	1	01
jal	1101111	1	11	0	0	10	0	xx

Tabla 2: Tabla de verdad de la unidad de control principal.

Nota crítica sobre el ALUControl: El `ALUControl` usa **4 bits**, no 3. El bit [3] diferencia ADD (0000) de SUB (1000). Si se truncara a 3 bits, la resta se interpretaría como suma y fallaría la operación `sub` o la lógica de comparación de `beq`.

2.7.2 Pseudocódigo de Decode

```

1  DECODE(clk, reset, StallD, FlushD, InstrF, PCF, PCPlus4F):
2    // 1. Registros IF/ID (flopenrc: con stall y flush)
3    always @(posedge clk):
4      if (reset || FlushD):
5        InstrD = 0; PCD = 0; PCPlus4D = 0;    // Insertar NOP
6      else if (!StallD):
7        InstrD = InstrF; PCD = PCF; PCPlus4D = PCPlus4F;
8
9    // 2. Extraer campos
10   Rs1D = InstrD[19:15]; Rs2D = InstrD[24:20]; RdD = InstrD[11:7];
11
12   // 3. Leer banco de registros (combinacional)
13   RD1D = (Rs1D != 0) ? registers[Rs1D] : 0;
14   RD2D = (Rs2D != 0) ? registers[Rs2D] : 0;
15
16   // 4. Generar senales de control
17   {RegWroteD, MemWroteD, ALUSrcD, ...} = decode_control(InstrD);
18
19   // 5. Extender inmediato
20   ExtImmD = extend_sign(InstrD[31:7], ImmSrcD);

```

Listing 4: Pseudocódigo de la etapa Decode

3 Etapa 3: Execute (EX)

La etapa Execute es el núcleo de cálculo del procesador. Es aquí donde la ALU (Arithmetic Logic Unit) procesa los datos y toma decisiones críticas para el flujo del programa.

Teoría Operativa: A diferencia de un procesador de ciclo único donde la decisión de saltar (Branch) se evalúa en Decode, en este pipeline se evalúa en Execute. La ALU realiza una resta (`sub`) entre los dos operandos y levanta la bandera `ZeroE` si son iguales. Esta bandera, al operarse mediante una compuerta AND con la señal `BranchE`, determina la señal `PCSrcE` (Branch Taken). Este retraso arquitectónico significa que el salto

no se resuelve hasta el tercer ciclo, lo que introduce una penalización de 2 ciclos (burbujas) en caso de que el salto sea tomado, los cuales son gestionados puramente por la *Hazard Unit*.

3.1 Componentes Activos

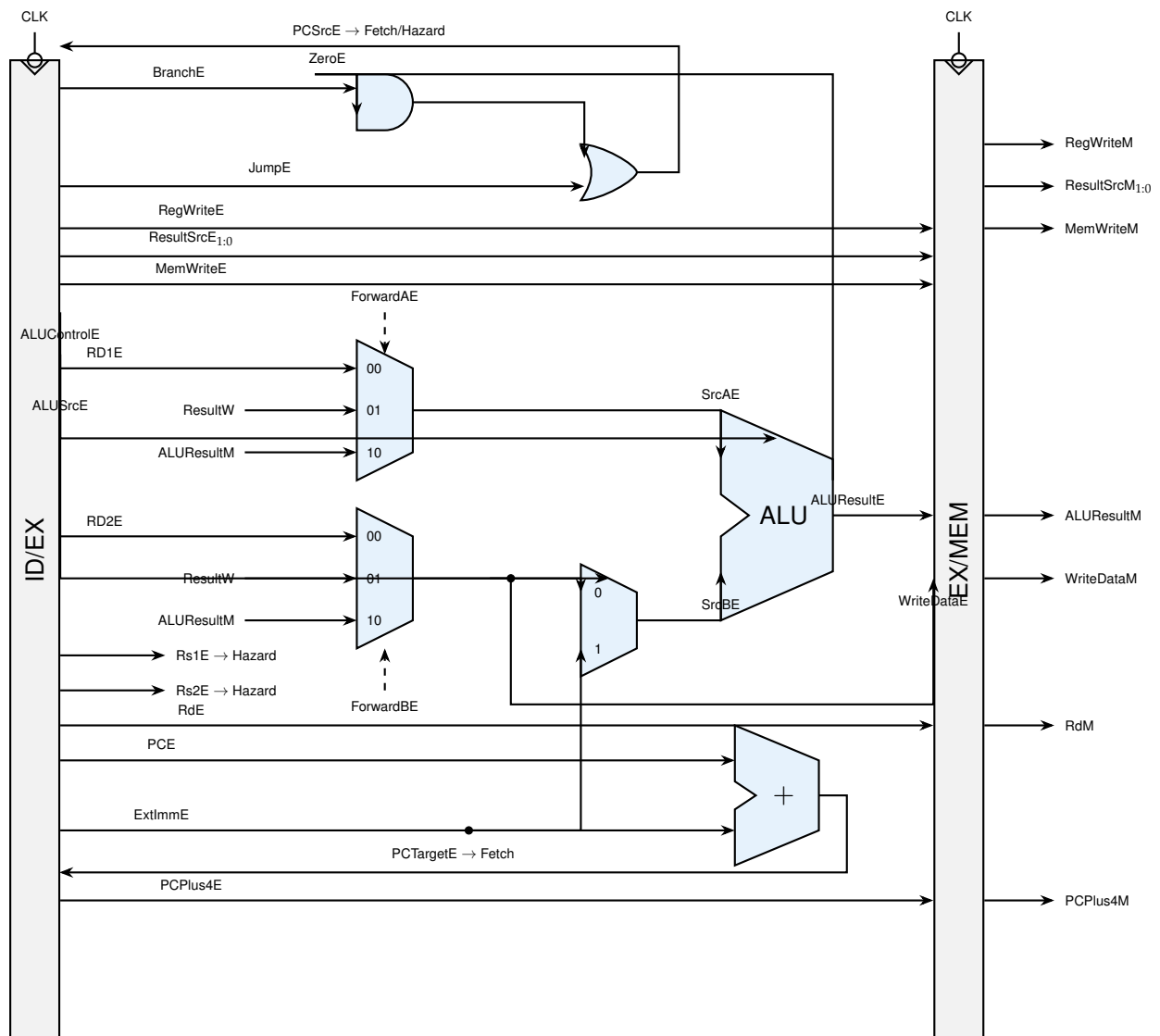


Figura 6: Etapa Execute. Muestra los multiplexores de Forwarding integrados antes de la ALU.

3.2 Pseudocódigo Completo del Execute

```
1 EXECUTE(clk, reset, FlushE, ForwardAE, ForwardBE,  
2         control_signals_D, data_D, ALUResultM, ResultW):  
3  
4 // 1. Registro ID/EX (flopenrc: sin stall, con flush)  
5 always @(posedge clk):
```

```

6   if (reset || FlushE):
7       all_signals = 0;          // Insertar burbuja NOP
8   else:
9       all_signals = decode_signals;
10
11  // 2. Forwarding Mux A
12  if (ForwardAE == 2'b10):      SrcAE = ALUResultM; // de Memory
13  else if (ForwardAE == 2'b01): SrcAE = ResultW;    // de Writeback
14  else:                        SrcAE = RD1E;        // del registro
15
16  // 3. Forwarding Mux B
17  if (ForwardBE == 2'b10):      WriteDataE = ALUResultM;
18  else if (ForwardBE == 2'b01): WriteDataE = ResultW;
19  else:                        WriteDataE = RD2E;
20
21  // 4. Selección del operando B de la ALU
22  if (ALUSrcE == 1): SrcBE = ExtImmE; // Inmediato
23  else:              SrcBE = WriteDataE; // Registro
24
25  // 5. Ejecutar ALU
26  ALUResultE = ALU(SrcAE, SrcBE, ALUControlE);
27  ZeroE = (ALUResultE == 0);
28
29  // 6. Calcular dirección de salto
30  PCTargetE = PCE + ExtImmE;
31
32  // 7. Decidir si se toma el salto
33  PCSrcE = (BranchE && ZeroE) || JumpE;

```

Listing 5: Pseudocódigo de la etapa Execute

4 Etapas Memory (MEM) y Writeback (WB)

Estas dos etapas finales son responsables de confirmar los resultados arquitectónicos, ya sea interactuando con la jerarquía de memoria o actualizando el banco de registros.

Teoría de Memoria y Sincronismo: La Memoria de Datos representa típicamente la caché L1 de datos en un procesador real. En nuestra implementación, la memoria es **síncrona para escritura y asíncrona para lectura**.

- **Escritura:** Solo ocurre en el flanco de subida del reloj cuando `MemWriteM` está en alto. Esto evita que señales inestables modifiquen el estado de la memoria.
- **Lectura:** Las memorias reales modernas suelen ser síncronas para lectura también (añadiendo latencia), pero nuestra implementación combinacional simplificada asume que el dato se lee instantáneamente si la dirección (`ALUResultM`) es válida.

4.1 Componentes Activos

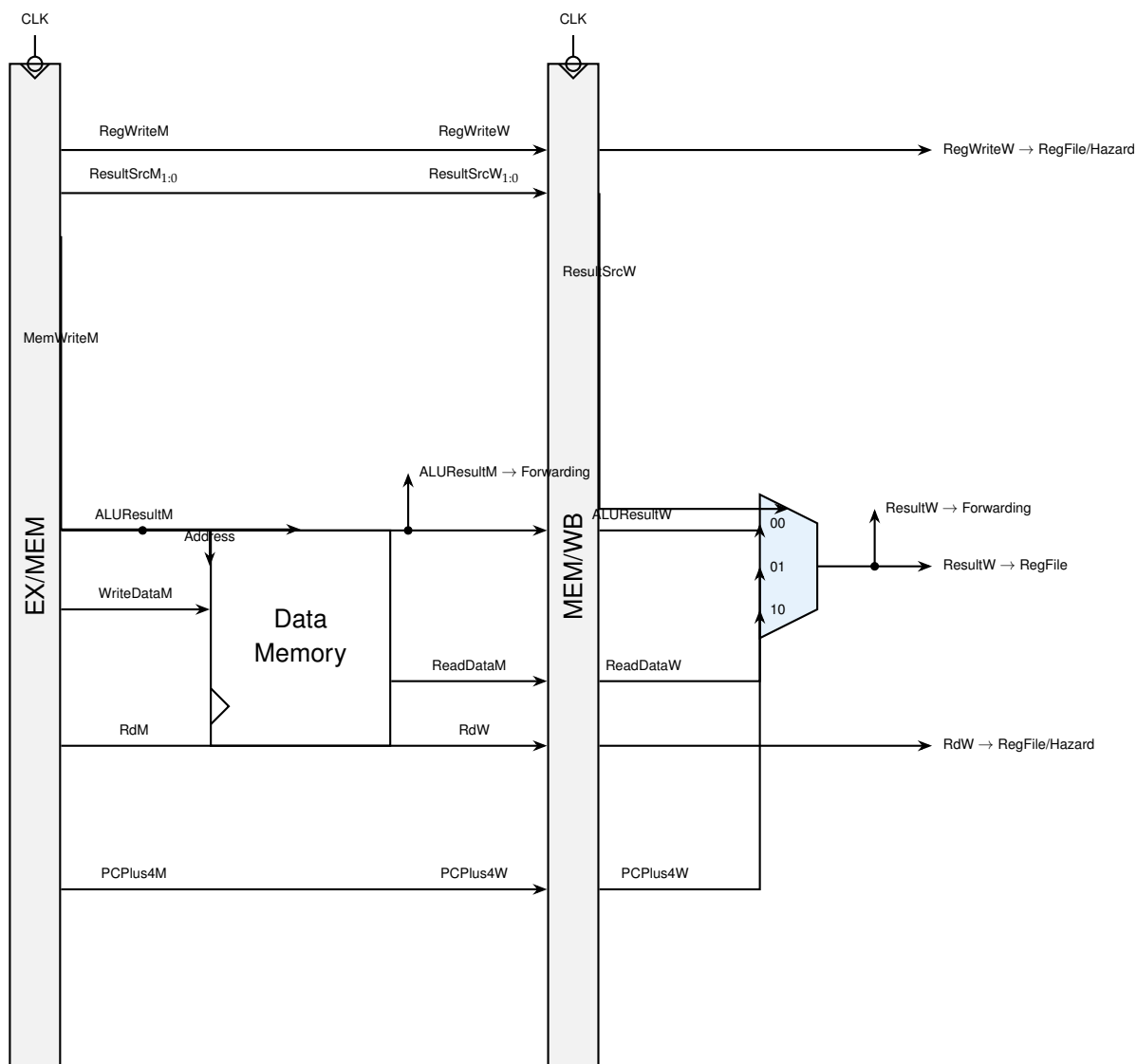


Figura 7: Etapas Memory y Writeback. La memoria de datos tiene escritura síncrona.

4.2 Pseudocódigo de ambas etapas

```

1 MEMORY(clk, reset, execute_signals):
2   // 1. Registro EX/MEM (flop: siempre avanza)
3   always @(posedge clk):
4     if (reset) all_signals = 0;
5     else      all_signals = execute_signals;
6
7   // 2. Acceso a Memoria de Datos
8   ReadDataM = data_memory[ALUResultM / 4]; // Lectura combinacional
9   if (MemWriteM == 1):                     // Escritura síncrona
10    data_memory[ALUResultM / 4] = WriteDataM;
11
12 WRITEBACK(clk, reset, memory_signals):
13   // 1. Registro MEM/WB (flop: siempre avanza)

```

```

14: always @(posedge clk):
15:     if (reset) all_signals = 0;
16:     else      all_signals = memory_signals;
17:
18: // 2. Seleccionar resultado final
19: if (ResultSrcW == 2'b00): ResultW = ALUResultW; // Aritmetica
20: else if (ResultSrcW == 2'b01): ResultW = ReadDataW; // Load (lw)
21: else if (ResultSrcW == 2'b10): ResultW = PCPlus4W; // JAL

```

Listing 6: Pseudocódigo de Memory y Writeback

5 Programa ISA sin Dependencias (1.0 pts)

Este programa prueba que las instrucciones básicas del procesador (addi, add, sw, lw) funcionan correctamente cuando **no existen dependencias de datos entre instrucciones consecutivas**. Para lograrlo, se insertan instrucciones nop (no-operation) entre cada instrucción real, dando tiempo suficiente para que los datos se escriban en los registros antes de ser leídos por la siguiente instrucción.

5.1 Código Ensamblador

```

1 00: addi x2, x0, 5      // x2 = 5
2 04: nop                // Separador
3 08: nop
4 0C: nop
5 10: add  x3, x2, x2     // x3 = 10
6 14: nop
7 18: nop
8 1C: nop
9 20: sw   x3, 100(x0)    // MEM[100] = 10
10 24: nop
11 28: nop
12 2C: nop
13 30: lw   x4, 100(x0)   // x4 = MEM[100] = 10
14 34: nop
15 38: nop
16 3C: nop
17 40: add  x5, x4, x3     // x5 = 10 + 10 = 20
18 44: sw   x5, 200(x0)   // MEM[200] = 20

```

Listing 7: Programa 1: ISA sin dependencias (test_isa_sin_dependencias.mem)

5.2 Waveform del Programa 1

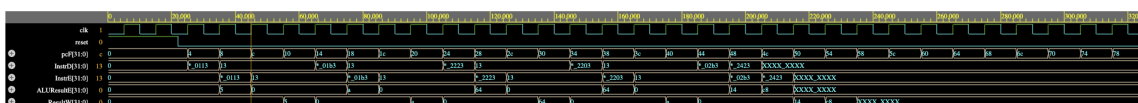


Figura 8: Waveform del Programa 1 (ISA sin dependencias). Las señales ForwardAE, ForwardBE, StallD y FlushE permanecen en 0 durante toda la ejecución.

5.3 Mostrar con Resultados Intermedios que el Programa Funciona Correctamente

Al observar el waveform, se verifica que:

- La señal `ALUResultE` muestra el valor `0x05` cuando `addi x2, x0, 5` pasa por `Execute`, y `0x0A` cuando `add x3, x2, x2` lo hace.
- La señal `ResultW` confirma que el valor `0x14` (20 en decimal) llega correctamente al final del pipeline como resultado de `add x5, x4, x3`.
- Las señales de la Hazard Unit (`ForwardAE`, `ForwardBE`, `StallD`, `FlushE`) permanecen en 0, demostrando que no se activó ningún mecanismo de protección. El programa fluye sin interrupciones.

6 Teoría de Hazard Unit y Evidencia de Fallos (0.5 pts)

6.1 Teoría de Control de Riesgos en Procesadores Segmentados

Al segmentar la ejecución de instrucciones, se solapan las etapas del datapath, rompiendo la ejecución estrictamente secuencial. Esto puede dar lugar a **riesgos** (hazards) que comprometen la integridad de los resultados si no se resuelven mediante hardware especializado:

- **Riesgos Estructurales (Structural Hazards):** Ocurren cuando dos instrucciones en etapas diferentes intentan acceder al mismo recurso físico al mismo tiempo. En este procesador se resuelven mediante una *Arquitectura Harvard*, que separa físicamente la memoria de instrucciones de la memoria de datos.
- **Riesgos de Datos (Data Hazards):** Surgen cuando una instrucción depende del resultado de una instrucción previa que aún no ha completado su escritura en el Register File. El caso más frecuente es la dependencia RAW (Read After Write).
- **Riesgos de Control (Control Hazards):** Producidos por instrucciones de salto condicional (`beq`) o absoluto (`jal`). Al leer la instrucción en `Fetch`, el procesador continúa cargando secuencialmente. Si el salto se resuelve como tomado en `Execute`, las instrucciones cargadas especulativamente en `Fetch` y `Decode` son incorrectas y deben descartarse.

6.2 Evidencia: Sin Hazard Unit los Programas Fallan

Para demostrar de forma empírica la necesidad crítica de la Hazard Unit, se procedió a desactivar el control de riesgos utilizando el módulo alternativo `hunit_disabled.v`. Este fuerza a cero los reenvíos y stalls. A continuación, se muestra el comportamiento de los programas de prueba bajo esta condición errónea:

6.2.1 Programa 2 SIN Hazard Unit (Forwarding)

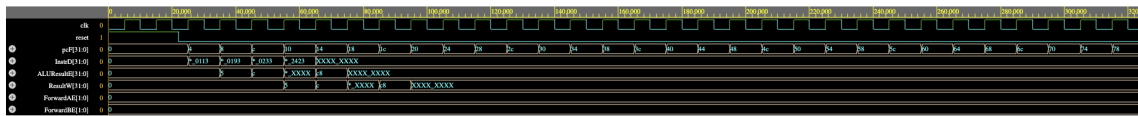


Figura 9: Programa 2 SIN Hazard Unit. La ALU opera con datos vacíos (0x00) porque los registros x2 y x3 aún no se han escrito cuando el `add` los necesita.

Interpretación: Sin forwarding, la instrucción `add x4, x2, x3` lee los registros x2 y x3 directamente del banco de registros. Como estas instrucciones previas (`addi x2` y `addi x3`) aún no han llegado a Writeback, la ALU recibe ceros y el resultado es incorrecto.

6.2.2 Programa 3 SIN Hazard Unit (Stalling)

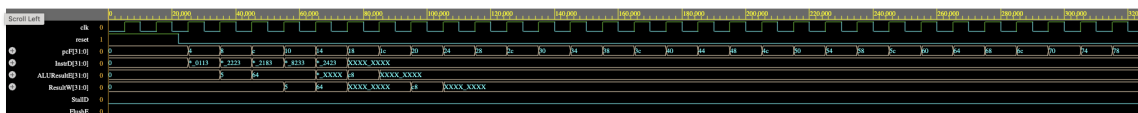


Figura 10: Programa 3 SIN Hazard Unit. El `add x4, x3, x2` usa un valor de x3 que aún no ha salido de la memoria.

Interpretación: El `lw x3, 100(x0)` carga un dato de memoria, pero el pipeline no se detiene. La instrucción `add x4, x3, x2` entra a Execute antes de que el dato esté disponible, produciendo un resultado corrupto.

6.2.3 Programa 4 SIN Hazard Unit (Flushing)

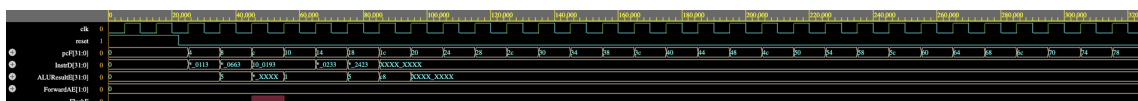


Figura 11: Programa 4 SIN Hazard Unit. Sin forwarding, las instrucciones previas al salto calculan incorrectamente, afectando la comparación del `beq`.

Interpretación: Sin la Hazard Unit, las instrucciones que preparan los operandos del `beq` no reciben los datos correctos vía forwarding. Esto puede causar que el salto se tome (o no) de forma errónea, ejecutando código que no debería ejecutarse.

7 Implementación del Hazard Unit (1.0 pts)

7.1 Descripción de Cambios en el Código

Para gestionar los riesgos de datos y de control de forma transparente para el programador (sin tener que inyectar NOPs manuales en el ensamblador), se implementó el módulo `hunit` (Hazard Unit) en el archivo `Hazard Unit/hunit.v`. Los principales cambios e implementaciones en el código incluyen:

1. **Diseño de la Unidad de Control de Riesgos:** Se diseñó un módulo puramente combinacional que monitorea las señales del datapath de forma continua y genera señales de bypass y de control en las etapas críticas.
2. **Estrategia de Pruebas de Fallo sin Dañar el Diseño:** Con el fin de demostrar la necesidad de la Hazard Unit sin modificar el procesador, se creó la unidad `hunit_disabled.v`. Este archivo tiene la misma firma (entradas y salidas) que la unidad real, pero inyecta un estado inactivo permanente (forzando reenvíos y stalls a 0), excepto por la redirección de saltos básicos para evitar que el PC entre en un ciclo infinito. La alternancia entre la simulación correcta y la errónea se realiza modificando una sola palabra en la instanciación del top-level (`riscv_pipeline.v`).

A continuación, se presentan los tres mecanismos principales de la unidad junto con sus diagramas y ecuaciones lógicas:

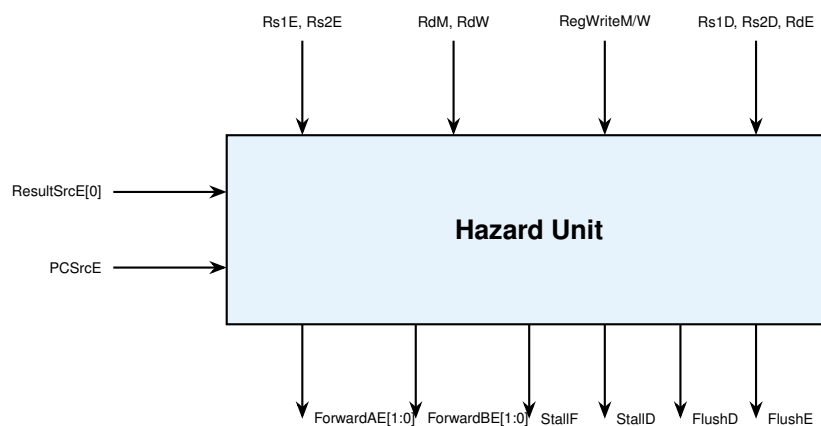


Figura 12: Diagrama de la Hazard Unit y sus múltiples conexiones.

7.2 Mecanismo 1: Forwarding (Reenvío de Datos)

El forwarding (o bypassing) es la solución de hardware más elegante para resolver los **Data Hazards** tipo RAW. Teóricamente, un resultado se calcula en la etapa Execute (ALU), pero arquitectónicamente no se escribe en el Register File hasta la etapa de Writeback (2 ciclos después). Si una instrucción posterior necesita este resultado inmediatamente, no puede leer el registro porque contiene un valor "viejo".

La Hazard Unit detecta que el registro destino de una instrucción en MEM o WB (*RdM* o *RdW*) coincide con los registros fuente de la instrucción actualmente en Execute (*Rs1E* o *Rs2E*). Al detectarlo, **cortocircuita** los multiplexores para inyectar el dato recién salido de la ALU (o de la Memoria) directamente en la entrada de la ALU de la instrucción actual, saltándose la espera por el Writeback.

```

1 // Forwarding para operando A (SrcAE)
2 if ((Rs1E == RdM) && RegWriteM && (Rs1E != 0)):
3     ForwardAE = 2'b10;    // Tomar resultado de Memory
4 else if ((Rs1E == RdW) && RegWriteW && (Rs1E != 0)):
5     ForwardAE = 2'b01;    // Tomar resultado de Writeback
6 else:
7     ForwardAE = 2'b00;    // Sin forwarding (usar registro)
8
9 // Forwarding para operando B (misma logica con Rs2E)

```

Listing 8: Lógica de Forwarding

7.3 Mecanismo 2: Stall (Load-Use Hazard)

El forwarding no puede resolver el Load-Use hazard (cuando un *lw* es seguido inmediatamente por una instrucción que lo usa). En ese caso, la Hazard Unit congela (stall) el PC y el IF/ID, e inserta una burbuja (flush) en el ID/EX.

```

1 // ResultSrcE[0] == 1 indica que la instruccion es un lw
2 lwStall = ResultSrcE[0] && ((Rs1D == RdE) || (Rs2D == RdE));
3
4 StallF = lwStall;    // Congelar PC (no avanza)
5 StallID = lwStall;   // Congelar IF/ID (mantener instruccion)
6 FlushE = lwStall;    // Vaciar ID/EX (insertar burbuja NOP)

```

Listing 9: Lógica de detección de Load-Use Hazard

7.4 Mecanismo 3: Flush (Control Hazard por Saltos)

Cuando se toma un salto (*PCSrcE*=1), las instrucciones que entraron especulativamente al Fetch y Decode deben descartarse, vaciando sus registros.

```

1 FlushD = PCSrcE;    // Vaciar IF/ID (descarta especulativa)
2 FlushE = lwStall || PCSrcE; // Vaciar ID/EX

```

Listing 10: Lógica de flush por salto tomado

7.5 Especificación de Señales de Control de Riesgos

Problema	Solución	Señales	Penalización
Data Hazard (R-type)	Forwarding	ForwardAE, ForwardBE	0 ciclos
Load-Use Hazard	Stall + Forwarding	StallF, StallID, FlushE	1 ciclo
Control Hazard	Flush	FlushD, FlushE	2 ciclos

7.6 Diagrama de Datapath Final (con Hazard Unit)

El datapath completo con todas las compuertas de control de riesgos, los multiplexores de forwarding para SrcAE y SrcBE, y las señales de stall y clear de los registros del pipeline se muestra a continuación:

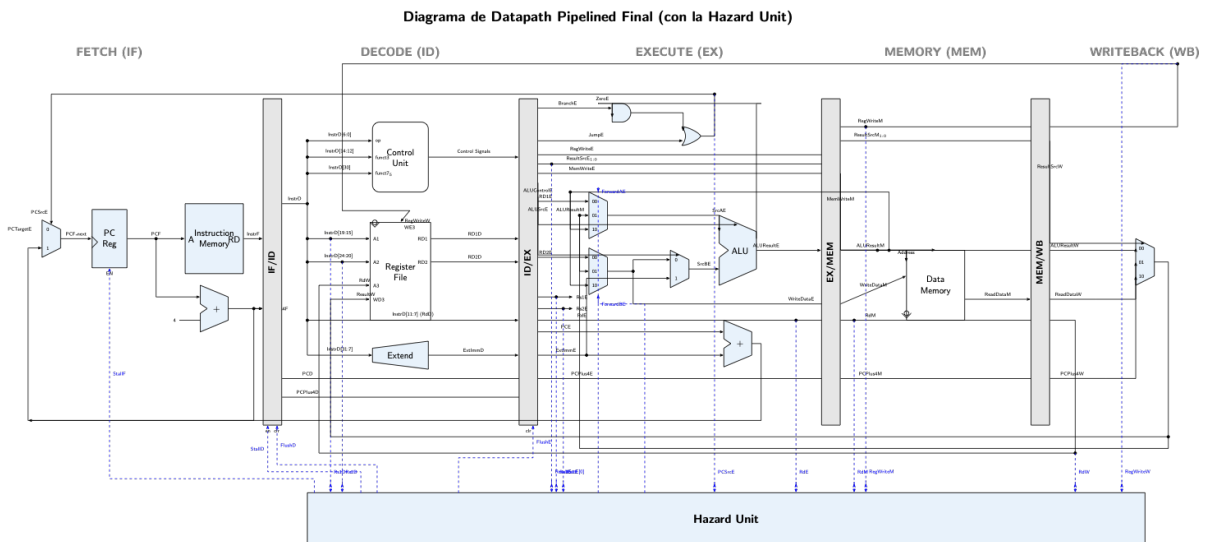


Figura 13: Diagrama del Datapath Pipelined completo del procesador (incluyendo la Hazard Unit).

El datapath final funciona correctamente ante cualquier dependencia de datos o de control debido a la incorporación de la Hazard Unit (unidad de control de riesgos) y los multiplexores de reenvío en la etapa Execute. La Hazard Unit supervisa continuamente y de forma combinacional los registros fuentes ($Rs1D$, $Rs2D$, $Rs1E$, $Rs2E$) y los compara con los registros de destino de instrucciones previas en las etapas de Execute (RdE), Memory (RdM) o Writeback (RdW) para tomar las siguientes acciones de control:

- **Forwarding (Reenvío):** Si hay un riesgo de datos del tipo RAW (Read After Write), la unidad activa las señales de selección $ForwardAE$ y/o $ForwardBE$. Esto hace que los multiplexores de 3 entradas ante la ALU tomen el dato más reciente desde Memory ($ALUResultM$) o Writeback ($ResultW$) en lugar de leer el valor obsoleto del banco de registros, logrando resolver el riesgo con 0 ciclos de penalización.
- **Stalling (Detección de Load-Use):** Cuando una instrucción de carga de memoria (lw) es seguida por otra que requiere su resultado en Execute, el reenvío no es suficiente ya que el dato se lee de memoria al final del ciclo de Memory. La unidad detecta esta condición y activa las señales de congelamiento $StallF$ y $StallD$ para detener la actualización del PC y del registro IF/ID por 1 ciclo, al mismo tiempo que activa $FlushE$ para inyectar una burbuja (instrucción NOP) en la etapa Execute, permitiendo al lw obtener el dato y realizar el reenvío en el ciclo siguiente.

- **Flushing (Saltos Condicionales y Saltos Directos):** Si se determina que un salto es tomado en Execute ($PCSrcE = 1$), las instrucciones cargadas especulativamente en las etapas Fetch y Decode deben ser descartadas para evitar que se ejecuten. La Hazard Unit activa las señales `FlushD` y `FlushE` para borrar los registros `IF/ID` e `ID/EX` (convirtiéndolos en NOPs), perdiendo 2 ciclos de penalización pero garantizando la correcta ejecución del programa.

8 Programas de Prueba (CON Hazard Unit) (1.0 pts)

Al activar la Hazard Unit real (`hunit.v`), los tres programas producen resultados correctos.

8.1 Programa 2: Forwarding

El programa prueba que la Hazard Unit detecta dependencias RAW y reenvía datos desde Memory y Writeback hacia Execute.

```

1 00: addi x2, x0, 5      // x2 = 5
2 04: addi x3, x0, 12     // x3 = 12
3 08: add  x4, x2, x3     // x4 = 5 + 12 = 17 (Forwarding)
4 0C: sw   x4, 200(x0)    // MEM[200] = 17

```

Listing 11: Programa 2: test_forwarding.mem

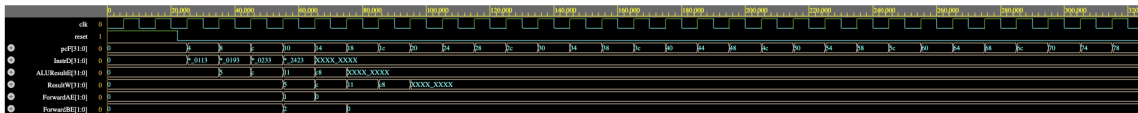


Figura 14: Programa 2 CON Hazard Unit. ForwardBE=10 reenvía x3 desde Memory; ForwardAE=01 reenvía x2 desde Writeback.

Interpretación: La Hazard Unit detecta que x3 (recién calculado, en etapa Memory) y x2 (en etapa Writeback) son necesarios por la instrucción `add x4, x2, x3`. Activa ForwardBE=10 y ForwardAE=01 para inyectar los valores correctos directamente a la ALU. El resultado es 0x11 (17 decimal), lo cual es correcto.

8.2 Programa 3: Stalling

El programa fuerza un Load-Use Hazard donde un `lw` es seguido inmediatamente por una instrucción que usa el dato cargado.

```

1 00: addi x2, x0, 5      // x2 = 5
2 04: sw   x2, 100(x0)    // MEM[100] = 5
3 08: lw   x3, 100(x0)    // x3 = MEM[100] = 5
4 0C: add  x4, x3, x2     // STALL: x3 aun no esta listo
5 10: sw   x4, 200(x0)    // MEM[200] = 10

```

Listing 12: Programa 3: test_stalling.mem

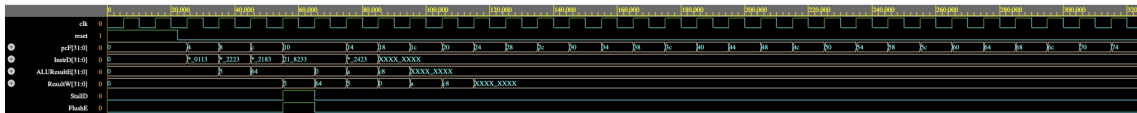


Figura 15: Programa 3 CON Hazard Unit. StallD congela Decode y Fetch durante 1 ciclo; FlushE inserta una burbuja en Execute.

Interpretación: La Hazard Unit detecta que el `lw x3` está en Execute (aún no ha accedido a memoria) y la instrucción `add x4, x3, x2` en Decode necesita `x3`. Activa `StallD=1` y `StallF=1` congelando el PC y el registro IF/ID por 1 ciclo. Simultáneamente, `FlushE=1` inserta una burbuja NOP en Execute. Al ciclo siguiente, el dato ya está disponible y se reenvía vía forwarding.

8.3 Programa 4: Flushing

El programa fuerza un Control Hazard con un salto condicional (`beq`) que es tomado.

```

1 00: addi x2, x0, 5          // x2 = 5
2 04: beq  x2, x2, 12         // Salta a linea 10 hex (siempre verdadero)
3 08: addi x3, x0, 1          // FLUSHED (no debe ejecutarse)
4 0C: addi x3, x0, 99         // FLUSHED (no debe ejecutarse)
5 10: add  x4, x0, x2         // x4 = 5 (destino del salto)
6 14: sw   x4, 200(x0)        // MEM[200] = 5

```

Listing 13: Programa 4: test_flushing.mem

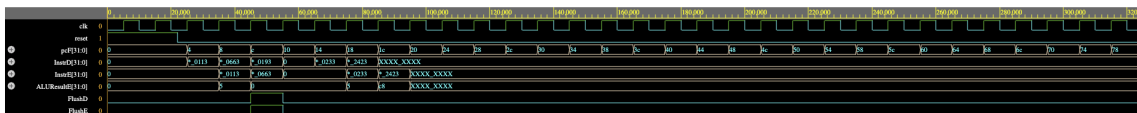


Figura 16: Programa 4 CON Hazard Unit. FlushE y FlushD se activan cuando `PCSrcE=1`, descartando las instrucciones especulativas.

Interpretación: El `beq x2, x2` se resuelve en la etapa Execute (ciclo 3). La ALU confirma que `x2 == x2` y activa `PCSrcE=1`. La Hazard Unit responde activando `FlushD` y `FlushE`, convirtiendo las instrucciones especulativas (`addi x3, x0, 1` y `addi x3, x0, 99`) en burbujas NOP (`0x00000000`). El PC salta directamente a `0x10` y la ejecución continúa correctamente.

9 Jerarquía de Archivos

Microarchitecture-Pipelined/

```

|-- riscv_pipeline.v          (Top module)
|-- tb.v                     (Testbench)
|-- run_sim.sh               (Script de simulacion)
|-- Datapath/
|   |-- Fetch/               fetch.v, imem.v

```

```

|  |-- Decode/      decode.v, regfile.v
|  |-- Execute/     execute.v
|  |-- Memory/      memory.v, dmem.v
|  |-- WriteBack/   writeback.v
|-- Controller/     controller.v
|-- Hazard Unit/    hunit.v, hunit_disabled.v
|-- Utils/          flopr.v, flopenr.v, flopenrc.v,
|                  mux2.v, mux3.v, adder.v, alu.v, extend.v
|-- Tests/
|  |-- test_isa_sin_dependencias.mem  (Programa 1)
|  |-- test_forwarding.mem            (Programa 2)
|  |-- test_stalling.mem              (Programa 3)
|  |-- test_flushing.mem              (Programa 4)
|-- Waves/
|  |-- waves_isa.vcd, waves_forwarding.vcd,
|  |   waves_stalling.vcd, waves_flushing.vcd
|  |-- waves_forwarding_NO_HAZARD.vcd,
|  |   waves_stalling_NO_HAZARD.vcd,
|  |   waves_flushing_NO_HAZARD.vcd
|-- Informe_LaTeX/  informe_pipeline.tex

```

10 Conclusiones

1. **Pipeline vs. Single-Cycle:** La arquitectura pipelined incrementa significativamente el rendimiento dividiendo la ejecución en 5 etapas solapadas, al coste de añadir complejidad con la Hazard Unit.
2. **Necesidad de la Hazard Unit:** Como se demostró en la Sección 4, sin la Hazard Unit los programas con dependencias producen resultados incorrectos. Los mecanismos de forwarding, stalling y flushing son indispensables para garantizar la correctitud del procesador.
3. **Forwarding:** Resuelve la mayoría de los Data Hazards sin penalización (0 ciclos perdidos), cortocircuitando resultados desde Memory y Writeback hacia Execute.
4. **Stalling:** El único caso que requiere detener el pipeline es el Load-Use Hazard, con una penalización mínima de 1 ciclo.
5. **Flushing:** Los saltos tomados provocan una penalización de 2 ciclos al descartar instrucciones especulativas, pero garantizan la integridad del flujo de control.
6. **Modularidad:** El diseño modular (cada etapa en su propio archivo) facilita la comprensión, depuración y extensión futura del procesador.